1) Display how many purchases and the total amount purchased by each customer using one SQL statement. [1]

```sql
SELECT
    CustomerID,
    COUNT(*) AS total_purchases,
    SUM(purchase_amount) AS total_amount
FROM purchases-- gets data from this table
GROUP BY CustomerID-- groups total purchases and total amount to each customer
```

Results | Messages

	CustomerID	total_purchases	total_amount
1	1	3	274.99
2	2	2	339.50
3	3	2	59.98
4	4	2	219.99
5	5	1	200.00

2) Display how many customers are in each city using one SQL statement. [1]

```sql
Select city, Count(CustomerID) as #_Of_Customers --grabs the city and number of customers
from Customer
Group by city-- groups rows that have the same value in the city column
```

Results | Messages

	city	#_Of_Customers
1	New Haven	13
2	Cape Coral	18
3	Springfield	11
4	Rockford	13
5	Daly City	10

3) Display all the sales each product is tied to, with the output sorted by product and category. [1]

4) Display all the products in each category. [1]

```sql
SELECT --grabs the information we want
    category,
    product_id,
    product_name
FROM products -- From the 'products' table
ORDER BY category, -- First, order by category name
         product_name; -- Then, order by product name within each category
```

Results | Messages

	category	product_id	product_name
1	Books	17	Cookbook
2	Books	16	Novel: Fiction
3	Electronics	2	Gaming Keyboard
4	Electronics	3	HD Monitor

5) Display the product name and category sorted by city, then by category, then by product. [1]

6) Display a count of the number of customers in each city sorted by city. [1]

```sql
SELECT
    city,                          -- The city of the customers
    COUNT(*) AS customer_count     -- The number of customers in each city
FROM customer                      -- From the 'customers' table
GROUP BY city                      -- Group the results by city
ORDER BY city;                     -- Sort the results by city in ascending order
```

89 %

Results | Messages

	city	customer_count
1	Aberdeen	12
2	Abilene	21
3	Akron	9
4	Albany	9
5	Albuquerque	11

7) Insert a new category "Health and Beauty Aids" with an appropriate description into the category table. (show me the sql and result set). [1]

```sql
INSERT INTO categories (category_name, description)-- adding new row
VALUES ('Health and Beauty Aids', 'Products related to personal care, wellness, and beauty.');
SELECT * FROM categories; --shows data in table
```

89 %

Results | Messages

	category_id	category_name	description
1	1	Health and Beauty Aids	Products related to personal care, wellness, and...

8) Insert records that indicates the purchases Judy Denth and Idris Elba made from Health and Beauty Aids and produce a report. Include the SQL to perform the inserts as well. [1]

9) Display the category, purchase date and amount for each recipient include LName, FName) for each category sorted by Lname that have purchased at least 4 products. [1]

10) Display a count of the number of products for each category using one SQL statement. [1]

G-Mans-Digital...- dbo.purchases | G-Mans-Digital-...- dbo.products | G-Mans-Digital-...- dbo.products | SQLQu

```sql
SELECT
    category,                      -- The category of the products
    COUNT(*) AS product_count      -- The number of products in each category
FROM products                      -- From the 'products' table
GROUP BY category                  -- Group the results by category
ORDER BY category;                 -- Sort the results by category in ascending order
```

89 %

Results | Messages

	category	product_count
1	Books	2
2	Electronics	3
3	Furniture	3
4	Home Appliances	2

****

Administrative tasks

11) Display the active logins in SQL Server. [1]

```sql
SELECT --information to display
    s.session_id,
    s.login_name,
    s.status,
    s.login_time,
    c.client_net_address,
    c.client_tcp_port,
    s.host_name,
    s.program_name
FROM
    sys.dm_exec_sessions s
JOIN
    sys.dm_exec_connections c
    ON s.session_id = c.session_id
WHERE
    s.is_user_process = 1  -- Filter to show only user sessions
ORDER BY
    s.login_time DESC;
```

89 %

Results | Messages

	session_id	login_name	status	login_time	client_net_address	client_tcp_port	host_name	program_name
1	72	G-MANS-DIGITAL-\fords	sleeping	2024-11-30 19:21:48.813	<local machine>	NULL	G-MANS-DIGITAL-	Microsoft SQL Server
2	59	NT SERVICE\SQLTELEMETRY$SQLEXPRESS01	sleeping	2024-11-30 19:18:24.500	<local machine>	NULL	G-MANS-DIGITAL-	SQLServerCEIP
3	52	G-MANS-DIGITAL-\fords	sleeping	2024-11-30 19:14:41.353	<local machine>	NULL	G-MANS-DIGITAL-	Microsoft SQL Server

12) Demonstrate how to terminate a connection to SQL Server. [1]

```sql
kill 80 --terminates connection 80
```

89 %

Messages

```
Commands completed successfully.

Completion time: 2024-11-30T19:25:44.3986441-05:00
```

13) Create a trigger for update, insert, delete on a table. Trigger writes message to an audit table that provides 'useful' information. [3]

```
-Mans-Digital-...oductIngredient    G-Mans-Digital-...oductIngredient*    SQLQuery3.sql -.
```

```sql
CREATE TABLE AuditLog (
    AuditID INT IDENTITY(1,1) PRIMARY KEY,
    ActionType VARCHAR(10),  -- 'INSERT', 'UPDATE', 'DELETE'
    TableName VARCHAR(50),   -- Name of the affected table
    UserName VARCHAR(50),    -- User who performed the action
    ChangeDate DATETIME,     -- Date and time of the change
    OldValue NVARCHAR(MAX),  -- Old value (for UPDATE/DELETE)
    NewValue NVARCHAR(MAX)   -- New value (for INSERT/UPDATE)
);
```

```
ns-Digital-...oductIngredient    G-Mans-Digital-...oductIngredient*    SQLQuery3.sql -...AL-\fords (80))*    SQLQuery2.sql -...AL-\for
```

```sql
CREATE TRIGGER trg_Audit_Customer
ON Customer
FOR INSERT, UPDATE, DELETE
AS
BEGIN
    -- Declare variables once
    DECLARE @OldValue NVARCHAR(MAX);
    DECLARE @NewValue NVARCHAR(MAX);

    -- Log INSERT operation
    IF EXISTS (SELECT * FROM inserted)
    BEGIN
        -- Get the new values from the inserted table
        SET @NewValue = (SELECT * FROM inserted FOR JSON PATH);  -- Get the inserted row as JSON

        -- Insert audit log for INSERT
        INSERT INTO AuditLog (ActionType, TableName, UserName, ChangeDate, NewValue)
        VALUES
        ('INSERT', 'Customer', SYSTEM_USER, GETDATE(), @NewValue);
    END

    -- Log DELETE operation
    IF EXISTS (SELECT * FROM deleted)
    BEGIN
        -- Get the old values from the deleted table
        SET @OldValue = (SELECT * FROM deleted FOR JSON PATH);  -- Get the deleted row as JSON

        -- Insert audit log for DELETE
        INSERT INTO AuditLog (ActionType, TableName, UserName, ChangeDate, OldValue)
        VALUES
        ('DELETE', 'Customer', SYSTEM_USER, GETDATE(), @OldValue);
    END

    -- Log UPDATE operation
    IF EXISTS (SELECT * FROM inserted) AND EXISTS (SELECT * FROM deleted)
    BEGIN
        -- Get both old and new values
        SET @OldValue = (SELECT * FROM deleted FOR JSON PATH);  -- Get the old row as JSON
        SET @NewValue = (SELECT * FROM inserted FOR JSON PATH); -- Get the new row as JSON

        -- Insert audit log for UPDATE
        INSERT INTO AuditLog (ActionType, TableName, UserName, ChangeDate, OldValue, NewValue)
        VALUES
        ('UPDATE', 'Customer', SYSTEM_USER, GETDATE(), @OldValue, @NewValue);
    END
END;
```

```sql
Insert Customer( CFName, CFLname)
Values ('Peter','Ban')

Update Customer
Set Phone = 1234567890
Where CustomerID = 2

Delete From Customer
Where CustomerID = 201

Select * from AuditLog
```

Results | Messages

AuditID	ActionType	TableName	UserName	ChangeDate	OldValue	NewValue
1	INSERT	Customer	G-MANS-DIGITAL-\fords	2024-11-30 19:38:58.297	NULL	[{"CustomerID":1,"CFName":"Gerrie","CFLname":"Kaa...
2	DELETE	Customer	G-MANS-DIGITAL-\fords	2024-11-30 19:38:58.300	[{"CustomerID":1,"CFName":"Gerrie","CFLname":"Ka...	NULL
3	UPDATE	Customer	G-MANS-DIGITAL-\fords	2024-11-30 19:38:58.300	[{"CustomerID":1,"CFName":"Gerrie","CFLname":"Ka...	[{"CustomerID":1,"CFName":"Gerrie","CFLname":"Kaa...

14) Create a trigger that performs a cascading delete in a second table when a record is deleted from a table.

15) SQL to backup a database. [1]

SQLQuery1.sql -...AL-\fords (64))*

```sql
BACKUP DATABASE [Experimental]
TO DISK = 'C:\Database Security\MSSQL16.SQLEXPRESS01\MSSQL\Backup\Experimental.bak'--location on disk
WITH FORMAT,--overwrites
     MEDIANAME = 'SQLServerBackups',--label for backup media
     NAME = 'Full Backup of Experitmental';--assigns description to backup
```

89 %

Messages

```
Processed 536 pages for database 'Experimental', file 'Experimental' on file 1.
Processed 2 pages for database 'Experimental', file 'Experimental_log' on file 1.
BACKUP DATABASE successfully processed 538 pages in 0.049 seconds (85.698 MB/sec).

Completion time: 2024-12-02T11:01:04.1660232-05:00
```

16) SQL to restore a database. [1]. Bonus point - back up a database on one server and restore on another. [1]

SQLQuery1.sql -...AL-\fords (64))*

```sql
RESTORE DATABASE [Experimental]
FROM DISK = 'C:\Database Security\MSSQL16.SQLEXPRESS01\MSSQL\Backup\Experimental.bak'--path to backup
WITH FILE = 1,
--Replace the paths with the desired storage location
     MOVE 'Experimental' TO 'C:\Database Security\MSSQL16.SQLEXPRESS01\MSSQL\DATA\Experimental.mdf',
     MOVE 'Experimental_log' TO 'C:\Database Security\MSSQL16.SQLEXPRESS01\MSSQL\DATA\Experimental_log.ldf',
     REPLACE;--overwrites existing database
```

89 %

Messages

```
Processed 536 pages for database 'Experimental', file 'Experimental' on file 1.
Processed 2 pages for database 'Experimental', file 'Experimental_log' on file 1.
RESTORE DATABASE successfully processed 538 pages in 0.042 seconds (97.656 MB/sec).

Completion time: 2024-12-02T11:11:49.8799706-05:00
```

17) SQL to create a user and assign to a role in a database. [1]

SQLQuery3.sql -...AL-\fords (52))* | SQLQuery1.sql -...AL-\fords (64))*

```sql
USE [Experimental];
GO
-- Create a login for server authentication
CREATE LOGIN [Manager2] WITH PASSWORD = 'I_<3_SQL';
GO
-- Create a database user linked to the login
CREATE USER [Peter] FOR LOGIN [Manager2];
GO
-- Assign the user to a role
ALTER ROLE [db_owner] ADD MEMBER [Peter];
GO
```

89 %

Messages

```
Commands completed successfully.
```

SQL Routines

18) Create an SQL routine that uses the Case statement. [1]

```sql
Select CFName,
Case
--When a customer name is 'Gerrie' it will say they are the 'Happiest Customer'
    When CFName = 'Gerrie' then 'Happiest Customer'
--When a customer name is 'Hadria' it will say they are a 'Banned Customer'
    When CFName = 'Hadria' then 'Banned Customer'
--Otherwise Customer Info will state they are a 'Allowed Customer'
    Else 'Allowed Customer'
End as 'Customer Info' -- will display text in a column called 'Customer Info'
From Customer
```

90 %

Results | Messages

	CFName	Customer Info
1	Gerrie	Happiest Customer
2	Hadria	Banned Customer
3	Nicoline	Allowed Customer
4	Celka	Allowed Customer

19) Create an SQL routine that uses nested IF/ELSE/IF [1]

SQLQuery1.sql -...AL-\fords (65))*

```sql
declare @@Number int -- declares the number varible
set @@Number = 17 --sets variable value to 17
if @@Number > 5-- If the variable "Number" is greater than 5 it will diplay the text below
    print('number is greater than 5')
else if @@Number < 5-- If the variable "Number" is less than 5 it will diplay the text below
    Print('number is less than 5')
else-- otherwise it will display the text below
    print('number is 5')
```

89 %

Messages

```
number is greater than 5
```

20) Create an SQL routine that uses at temp table. [2]

21) SQL that joins tables that are located in two databases. [1]

SQLQuery2.sql -...AL-\fords (76))* | SQLQuery1.sql -...AL-\fords (65))*

```sql
-- Select specific columns from the Customer and Promo tables.
SELECT
    c.CustomerID,  -- The unique identifier for each customer.
    c.CFName,      -- The first name of the customer.
    p.PromoID,     -- The unique identifier for each promotion.
    p.PrName       -- The name of the promotion.
FROM
    Coffee.dbo.Customer AS c -- Access the Customer table in the Coffee database, alias it as 'c'.
INNER JOIN
    Coffee2.dbo.Promo AS p   -- Access the Promo table in the Coffee2 database, alias it as 'p'.
ON
    c.CustomerID = p.PromoID -- Join the tables where the CustomerID matches the PromoID.
```

89 %

Results | Messages

	CustomerID	CFName	PromoID	PrName
1	1	Gerrie	1	BOGO
2	2	Hadria	2	Happy Hour Specials
3	3	Nicoline	3	Loyalty Card Program

22) SQL that joins tables that are located on two database servers. [1]

23) Create a table that includes a person's name, address, city, state/province, country. Must have at least 10,000 rows, no more than 5 duplicate names, at least 5 last names must have 100 but less than 250 occurrences. [3]

24) Create a phone number table that supports table created in #23 (at least 90% of records in table from 21 must have a unique phone number). [2]

25) Create an update routine that includes an SQL trigger that updates data in a second table. [2]

26) Create a table with at least 10 columns of data and 1,000,000 rows. Run a series of select statements without an index and record the execution time. Create an index and rerun the select statements previously used. Record the execution time and comment on the run time comparison. Realize execution plans can be cached, which may impact your results. Your series of select statements should include selecting one particular record, selecting a small number of records (10 or so), selecting multiple records that would be randomly distributed (such as a phone book - find Jones, Miller, Roberts, Williams), selecting about 10% of the records. [5]

27) Create a routine that captures errors (@@error) and useful information that can be utilized in a variety of circumstances. An error logging table should be included. [2]

SQLQuery1.sql -...AL-\fords (64))*

```sql
CREATE TABLE dbo.ErrorLog (
    ErrorLogID INT IDENTITY(1,1) PRIMARY KEY,--auto incremented
    ErrorNumber INT,--error number returned by system
    ErrorMessage NVARCHAR(4000),--description of error
    ErrorProcedure NVARCHAR(200),--Sql statement that generated error
    ErrorLine INT,--Line error occured
    ErrorDateTime DATETIME DEFAULT GETDATE(),--date and time error was logged
    UserName NVARCHAR(100) DEFAULT SYSTEM_USER,--who encountered error
    SessionID INT DEFAULT @@SPID--session ID for the connection that generated the error
);
```

89 %

Messages

Commands completed successfully.

SQLQuery1.sql -...AL-\fords (64))*

```sql
CREATE PROCEDURE dbo.LogError
AS
BEGIN
    DECLARE @ErrorNumber INT;
    DECLARE @ErrorMessage NVARCHAR(4000);
    DECLARE @ErrorProcedure NVARCHAR(200);
    DECLARE @ErrorLine INT;

    -- Capture error details
    SET @ErrorNumber = ERROR_NUMBER();
    SET @ErrorMessage = ERROR_MESSAGE();
    SET @ErrorProcedure = ERROR_PROCEDURE();
    SET @ErrorLine = ERROR_LINE();

    -- Insert error details into the ErrorLog table
    INSERT INTO dbo.ErrorLog (ErrorNumber, ErrorMessage, ErrorProcedure, ErrorLine)
    VALUES (@ErrorNumber, @ErrorMessage, @ErrorProcedure, @ErrorLine);
END
```

89 %

Messages

Commands completed successfully.

SQLQuery1.sql -...AL-\fords (64))*

```sql
BEGIN TRY
    -- Sample code that may cause an error
    DECLARE @Result INT;
    SET @Result = 1 / 0;  -- This will generate a divide-by-zero error
END TRY
BEGIN CATCH
    -- Log the error details
    EXEC dbo.LogError;
END CATCH
```

89 %

Messages

(1 row affected)

```sql
SELECT * FROM dbo.ErrorLog
```

89 %

Results | Messages

	ErrorLogID	ErrorNumber	ErrorMessage	ErrorProcedure	ErrorLine	ErrorDateTime	UserName	SessionID
1	1	8134	Divide by zero error encountered.	NULL	4	2024-12-02 11:46:11.747	G-MANS-DIGITAL-\fords	64

28) Create a routine that demonstrates characteristics of Union [1]

29) Create a routine that uses Select Into to populate a table from another table. [1] Extra point for randomizing. [1]

SQLQuery3.sql -...AL-\fords (80))* | SQLQuery2.sql -...AL-\fords (76))*

```sql
SELECT CustomerID, CFName, CFLname, City, CountryID -- grabs certain columns
INTO CustomerBackup -- creates new table
FROM Customer-- gets information from this table
WHERE CountryID = 1; -- grabs only information when they are from Canada
```

89 %

Results | Messages

```
(2474 rows affected)
```

30) SQL routine that produces a graphical output such as a chart. [3]

31) Create a SQL routine that uses a subquery. [1]

```sql
SELECT
    p.PName
FROM
    Product p
WHERE
    p.ProductID IN (
        -- Subquery to find products that have a greater calorie amount than the average
        SELECT i.ProductID
        FROM ProductIngredient i
        WHERE i.Ammount > (
            -- Subquery to calculate the average amount of calories
            SELECT AVG(Ammount) FROM ProductIngredient
        )
    );
```

89 %

Results | Messages

	PName
1	Americano
2	Cappuccino
3	Mocha

32) Capture the IP address of the connection and include it in your @@error error logging routine. [3]

33) Create a sql routine that uses a subquery (nested query). [2]

34) Create a sql routine that uses a Common Table Expression (CTE). [2]

35) Create a SSIS routine that imports an Excel file and is scheduled to run at 9am every day. [3]